

Study and Implementation of Git

Nandan Chandrasekaran

Department of Computer Science and
Engineering

Manipal Institute of Technology, MAHE
Manipal, India

nandan.mitmpl2026@learner.manipal.edu

Abstract – This task was carried out to understand the version control system Git [2]. The tutorials on the Learn Git Branching website [1] were used to study concepts like commits, branches, merging, rebasing and remote repositories. These concepts were then used on a demo repository containing the solution files of the problems.

Keywords—Git, version control, commits, branching, repositories, merging, rebasing, remote repositories

I. INTRODUCTION

Software projects undergo numerous changes during development making it very important to record previous versions of the project. Version control systems help us achieve this by providing a structured method for these changes and maintaining project history. Git is one such version control system that is used by most developers around the globe. Git is a distributed version control system that maintains repositories locally allowing for multiple operations to be performed without the need of being connected to a remote server continuously. Git provides features such as clone, commit, push, pull, branching, merging and rebasing for managing project history. This report is written to understand the underlying structures along with how these commands function.

This report is divided into two broader parts. Sections II to VII explain concepts like what a commit is and how they're stored, what a branch is, the difference between merging and rebasing, how push and pull work with a remote repository, how git log and git relog are different, and how merge conflicts are resolved. Section VIII is a demonstration that was performed on a local repository made for this task, where the Task 1 Python solutions were committed and pushed.

II. GIT COMMITS AND HISTORY

A. GIT Commits

A commit represents a saved version of a project at a particular point in time. Each commit references a previous commit which enables git to record the history of changes made to a project. When a new commit is created the current branch moves to the new commit and every new commit contains a hardcoded pointer to its parent commit. This is what allows Git to maintain history of the project without losing previous versions.

B. HEAD and Relative References

HEAD is the name GIT uses to reference the branch that is currently checked out. It normally points to the currently checked out branch, which in turn points to the latest commit created.

Git also allows previous commits to be referenced using something called relative references. For instance, HEAD^ and HEAD~<number> helps us move back through commits. These relative references are useful as you can then reference previous commits without using the full commit hash (which looks something like: 7af68a4128b936d85a11cda34a9e22303fb1462d).

III. BRANCHING AND BRANCH MANAGEMENT

A. What a Branch really is

A branch like main or feature is not a separate copy of the whole repository. It is just a small file inside .git that stores the hash code of one commit. So, creating a new branch, which is done with either git branch or git checkout -b <branch name>, is a very fast operation as all it does is create a new file containing the hash code in it. A branch is a pointer to a commit, creating a new commit moves that pointer forward as well.

B. Merging

Merging is the feature on Git that enables us to bring changes from one branch onto another branch. If the branch being merged into has not moved since the other branch was created, Git can do what is called a fast-forward merge, where it just moves the pointer forward and no new commit is created. If both branches have new commits of their own, Git does a three way merge instead where it looks at the last common commit of both branches, compares the changes made on each side, and creates one new commit that has two parents.

C. Rebasing

Another technique used to introduce changes made in one branch into another branch is rebasing. However, the process of rebasing is different from merging. For instance, running the command git rebase main will cause every commit that is unique for the current branch to be reapplied on top of the latest commit of the main branch, thus changing the hash value of the commit which is formed as a result of the rebase even though the file's content remains unchanged. As a result of that, we will get the history without a merge commit. As rebasing changes commit hashes, this technique must not be applied to the branch that was already pushed and is being used by other people, as it results in two different histories of the same changes.

IV. REMOTE REPOSITORIES : PUSH & PULL

A. Tracking branching and origin

A remote, usually named origin by most developers, is a shortcut name pointing to your remote repository on the internet. It also contains extra references like origin/main which keep track of where each branch was the last time the repository communicated to the remote. These remote tracking references only get updated when a fetch, pull or push is done, they do not change just because a normal commit is made locally.

B. Fetch vs Pull

`git fetch` downloads new commits from the remote and updates references like origin/main, it does not touch the files in the working directory, hence why it is also safe to run. On the other hand, `git pull` is basically a fetch followed by a merge of that updated remote branch into the current branch. Since pull also includes a merge, it can create a merge commit and can also run into a conflict the same way a normal merge can.

C. Force Pushing

A normal `git push` only works if it just adds new commits on top of what the remote already has. If the local branch was rebased or if changes were made to a commit, the local and remote history no longer match up, so a normal push gets rejected with an error and the remote is left unchanged.

`git push --force` can be used to push anyway and overwrite the remote branch. However, this can delete commits that someone else already pulled, so it should be used carefully.

V. HISTORY, LOGS AND THE REFERENCE LOG

A. git log and Commit Ancestry

`git log` follows the commit graph starting from HEAD and going backwards through the parent commits. Therefore, it shows the actual permanent history of the branch. Every commit shown by `git log` can be reached by following these parent links. Options like `--oneline`, `--graph` and `--all` make it easier to see the branch and merge structure on the screen.

B. git reflog

`git reflog` is different from `git log`. It is a local record of every place HEAD has pointed to, in the order it happened. So, it includes checkouts, commits, merges, rebases and resets. Unlike `git log`, the entries in `git reflog` do not have to be connected to each other, because commands like `reset` or `rebase` can leave behind old commits behind instead of building off of them. `reflog` entries only exist locally and are never pushed to a remote.

- `git log` shows the ancestry of the current commit.
- `git reflog` shows where HEAD has actually been, in the order it happened.

VI. RESOLVING MERGE CONFLICTS

A merge conflict happens when Git cannot automatically combine two branches during a merge because both of them have changes made to the same line of the same file. When this happens, Git stops and marks the conflicting part of the file with `<<<<<<< HEAD`, then

`=====`, and then `>>>>>>>` followed by the branch name.

To fix a conflict, the file has to be opened and edited manually. After that, `git add` is run on the file to mark it as resolved, and `git commit` is run to finish the merge.

VII. STASHING

`git stash` is used to temporarily save changes that have not been committed yet, without actually making a commit. A simply example of a use case scenario of `git stash` is, say you're halfway through building a feature on branch1 when a sudden bug arises that requires you to switch to branch2. Git won't let you normally switch because your current file gets overwritten. Stashing helps you get a clean environment that lets you switch and fix the bug.

`git stash list` shows all the saved stashes. `git stash pop` brings back the most recent one and removes it from the list. `git stash apply` brings it back but keeps it saved just in case it is needed again.

VIII. DEMONSTRATION : LEARNGITBRANCHING AND A LOCAL REPOSITORY

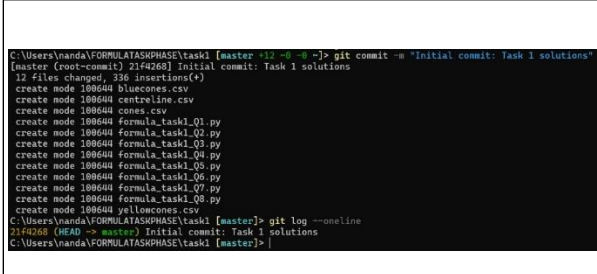
A. A little about the Main and Remote tabs on Learn Git Branching

The Main tab helped me understand the basic Git commands and how commits and branches are managed on local repositories whereas the Remote tab gave me a better understanding of pushing, pulling, and working with remote branches. I found the difference between merge and rebase especially useful, as they result in different commit histories. I also found the diverged remote exercises useful, especially seeing how Git can reject a push when the local and remote histories are no longer in sync.

B. Local Repository setup and Task 1 integration

I created a repository `git init` inside a folder containing the eight Task 1 Python solution files. All the files were staged with `git add .` and recorded in a single initial commit using `git commit`.

I then created an empty repository GitHub [3] and linked to the local repository as origin using `git remote add origin`, after which the initial commit was pushed using `git push -u origin main`.



```
C:\Users\nanda\FORMULATASKPHASE\task1 [master +12 -0 -0 -]> git commit -m "Initial commit: Task 1 solutions"
[master (root-commit) 21f4268] Initial commit: Task 1 solutions
12 files changed, 336 insertions(+)
create mode 100644 bluecones.csv
create mode 100644 centreline.csv
create mode 100644 cones.csv
create mode 100644 formula_task1_Q1.py
create mode 100644 formula_task1_Q2.py
create mode 100644 formula_task1_Q3.py
create mode 100644 formula_task1_Q4.py
create mode 100644 formula_task1_Q5.py
create mode 100644 formula_task1_Q6.py
create mode 100644 formula_task1_Q7.py
create mode 100644 formula_task1_Q8.py
create mode 100644 yellowcones.csv
C:\Users\nanda\FORMULATASKPHASE\task1 [master]# git log --oneline
21f4268 (HEAD -> master) Initial commit: Task 1 solutions
C:\Users\nanda\FORMULATASKPHASE\task1 [master]#
```

Fig. 1 Output of `git log --oneline` confirming the initial commit.

C. Demonstrating Commit, Branch, and Merge

I created a new branch named feature-update using `git checkout -b`. I then added a feature to the Task 1 Question 1 python file (output dictionary arranges the

output alphabets in alphabetical order) and committed on this branch. Switching back to main and running `git merge feature-update` completed the merge. Since main had not changed since the branch was created, this resulted in a fast-forward merge, meaning Git just moved the main branch pointer forward with no separate merge commit created, as explained in Section III.B.

```
C:\Users\Nanda\FORMULATASKPHASE\task1 [main #]> git checkout -b feature-update
Switched to a new branch 'feature-update'
C:\Users\Nanda\FORMULATASKPHASE\task1 [feature-update #]> git add
C:\Users\Nanda\FORMULATASKPHASE\task1 [feature-update #]> git commit -m "Update: output dictionary sorts the output in alphabetical order"
1 file changed, 6 insertions(+), 6 deletions(-)
C:\Users\Nanda\FORMULATASKPHASE\task1 [feature-update #]> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
C:\Users\Nanda\FORMULATASKPHASE\task1 [main #]> git merge feature-update
Merging 21f4268..08017a3
git-forward
 formula_task1_Q2.py | 12 +++++-
 1 file changed, 6 insertions(+), 6 deletions(-)
C:\Users\Nanda\FORMULATASKPHASE\task1 [main #]> git log --oneline --graph --all
* 08017a3 (feature-update) Update: output dictionary sorts the output in alphabetical order
* 21f4268 (origin/main) Initial commit: Task 1 solutions
C:\Users\Nanda\FORMULATASKPHASE\task1 [main #]>
```

Fig. 2 resulting commit graph using `git log --oneline --graph --all`.

D. Demonstrating a Merge Conflict and Resolution

To demonstrate a real merge conflict, I made a branch named `conflict-branch` and the first line of one Task 1 file was edited (with just a demo comment) and committed on it. The same line of the same file was then edited differently on main and committed there as well, so that both branches had changed the identical line since their common ancestor. Running `git merge conflict-branch` on main produced a conflict, and Git marked the file with `<<<<<<<`, `=====`, and `>>>>>>` markers as shown below.

```
1 <<<<<< HEAD (Current Change)
2 # version from main
3 =====
4 # version from conflict-branch
5 >>>>>> conflict-branch (Incoming Change)
```

Fig. 3, Separating main's version from conflict-branch's version.

```
C:\Users\Nanda\FORMULATASKPHASE\task1 [main #]> git log --oneline --graph
* dae737f (HEAD -> main) Resolve merge conflict on line 1
* 42be0b3 (conflict-branch) Change line 1 on conflict-branch
* 1 d6e4650 Change line 1 on main
* 08017a3 (feature-update) Update: output dictionary sorts the output in alphabetical order
* 21f4268 (origin/main) Initial commit: Task 1 solutions
```

Fig. 4, Shows the merge commit

The file was manually edited to remove the markers and keep the intended line, and the conflict was marked resolved using `git add` followed by `git commit`, completing the merge as shown in Fig. 4.

E. Demonstrating Push and Pull

After the merge, I pushed the main updated branch to the remote using `git push origin main`. To demonstrate a pull, a small change was made directly on GitHub's website (random test comment) and committed there, so that the remote repository had a commit the local repository did not yet have. When I ran `git pull origin main`, locally it fetched this new commit and merged it into the local main branch. Since the local branch had not diverged from the remote at that point, this was a fast-forward pull, and the change appeared immediately in the local files, as shown in Fig. 5.

```
C:\Users\Nanda\FORMULATASKPHASE\task1 [main #]> git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 997 bytes | 43.00 KiB/s, done.
From https://github.com/NandanChandrasekaran/git-task-report
 * branch          main       -> FETCH_HEAD
 * dae737f..3f0cfce main       -> origin/main
Updating dae737f..3f0cfce
Fast-forward
 formula_task1_Q2.py | 1 +
 1 file changed, 1 insertion(+)
```

Fig. 5, Fast-forward pull

F. Log and Reflog Evidence

Fig. 6 shows the output of `git reflog` for the repository, listing every checkout, commit, and merge performed during this demonstration in the order it happened, each entry referenced as `HEAD@{n}`. Comparing this with the output of `git log --oneline --graph --all`, shown earlier, illustrates the distinction discussed in Section V: `git log` shows only the permanent ancestry of commits reachable from the current branch, whereas, `git reflog` shows the complete local movement history of `HEAD`, including any actions that did not add new ancestry to the branch.

```
C:\Users\Nanda\FORMULATASKPHASE\task1 [main #]> git reflog
3f0cfce (HEAD -> main, origin/main) HEAD@{0}: pull origin main: fast-forward
dae737f HEAD@{1}: commit (merge): Resolve merge conflict on line 1
d6e4650 HEAD@{2}: commit: Change line 1 on main
08017a3 (feature-update) HEAD@{3}: checkout: moving from conflict-branch to main
42be0b3 (conflict-branch) HEAD@{4}: commit: Change line 1 on conflict-branch
08017a3 (feature-update) HEAD@{5}: checkout: moving from main to conflict-branch
08017a3 (feature-update) HEAD@{6}: checkout: moving from main to main
08017a3 (feature-update) HEAD@{7}: merge feature-update: fast-forward
21f4268 HEAD@{8}: checkout: moving from feature-update to main
08017a3 (feature-update) HEAD@{9}: checkout: moving from main to feature-update
21f4268 HEAD@{10}: checkout: moving from main to feature-update
21f4268 HEAD@{11}: Branch: renamed refs/heads/master to refs/heads/main
21f4268 HEAD@{12}: commit (initial): Initial commit: Task 1 solutions
```

Fig. 6, Output of `git log --oneline --graph --all`

IX. CONCLUSION

While doing this assignment, it has been easy to realize that Git is a system based on the principle of commits, branches, and pointers, which shift when commands are executed. After completing the exercises from *Learn Git Branching*, it became easy to understand visually the notion of branching, merging, and rebasing, and furthermore, repetition of the same operations on a local repository linked to GitHub remotely made it possible to see how all these concepts can be used in practice and gave me an understanding of how software development is performed within teams. Performing a merge conflict intentionally was especially helpful since it has allowed me to see how Git detects such conflicts and resolves them manually.

ACKNOWLEDGMENT

I would like to thank the Driverless Subsystem Seniors for providing the guidance and resources that helped me complete the task with a deeper understanding of the version control system Git and the concepts discussed in this report.

REFERENCES

- [1] *Learn Git Branching*. [Online]. Available: <https://learngitbranching.js.org/>
- [2] Git, "Git Reference" [Online]. Available: <https://git-scm.com/docs>
- [3] GitHub Docs, "Quickstart," GitHub, Inc. [Online]. Available: <https://docs.github.com/en/get-started/quickstart>